

JAVA

Complete Interview & Exam Guide

From Basics to Advanced — Every Topic, Every Question

Topics Covered: Introduction • Data Types • Operators • Control Statements • Arrays • OOP • Inheritance • Polymorphism • Abstraction • Encapsulation • Exception Handling • Strings • Collections • Generics • Java 8 • Multithreading • I/O • Memory • Design Patterns • Modern Java

Table of Contents

Topic 1 — Introduction to Java & the main Method

Section 1: What is Java?

Q1. What is Java?

Java is a high-level, class-based, object-oriented programming language. It follows the **WORA** principle — Write Once, Run Anywhere.

Q2. Who created Java and when?

Created by **James Gosling** at Sun Microsystems, released in **1995**. Now owned by Oracle.

Q3. What are the main features of Java?

Simple · Object-Oriented · Platform Independent · Secure · Robust · Multithreaded · Portable · High Performance · Distributed · Dynamic

Q4. Is Java compiled or interpreted?

Both. Java source code is compiled by `javac` into bytecode (.class), then the JVM interprets or JIT-compiles that bytecode at runtime.

Q5. Difference between JDK, JRE, and JVM?

JVM — executes bytecode; platform-specific **JRE** — JVM + libraries needed to run Java programs **JDK** — JRE + compiler + tools to develop Java programs

Q6. What is bytecode?

Platform-independent intermediate code generated by the Java compiler. The JVM reads and executes it on any platform.

Q7. Is Java 100% object-oriented?

No. Java uses primitive data types (`int`, `char`, `boolean`, etc.) which are not objects. Wrapper classes exist for all primitives.

Section 2: The main Method — Signature & Structure

Q8. What is the signature of the main method?

```
public static void main(String[] args)
```

Q9. Why is main declared public?

So the JVM (outside your class) can call it without access restrictions.

Q10. Why is main declared static?

The JVM calls `main` **without creating an object** of the class. Static methods belong to the class itself.

Q11. Why is the return type void?

`main` does not return any value to the JVM. Use `System.exit(code)` to return an exit code.

Q12. What is String[] args?

An array of `String` holding **command-line arguments**. Example: `java MyApp Hello World` → `args[0]="Hello", args[1]="World"`.

Q13. Can we use String... args (varargs) instead?

Yes. `public static void main(String... args)` is valid. Varargs internally creates an array.

Section 3: Can main be Overloaded, Overridden, or Final?

Q14. Can the main method be overloaded?

Yes. Multiple `main` methods with different parameters are allowed. Only `main(String[] args)` is called by JVM as entry point.

```
public static void main(String[] args) { main(10); } // JVM entry
public static void main(int x) { System.out.println(x); } // overloaded
```

Q15. Can the main method be overridden?

No. Since `main` is static, it cannot be overridden — only **hidden** (method hiding). JVM always calls the class's own `main`.

Q16. Can main be declared final?

Yes. `public static final void main(String[] args)` is valid and JVM recognizes it as entry point.

Q17. Can main be declared synchronized?

Yes. `public static synchronized void main(String[] args)` is valid.

Q18. Can main be declared abstract?

No. `abstract` and `static` cannot be combined in Java.

Q19. Can main throw exceptions?

Yes. `public static void main(String[] args) throws Exception` is valid. JVM catches uncaught exceptions and prints stack trace.

Section 4: What happens if main is Missing or Modified?

Q20. What happens if main is not public?

Compiles fine but throws `Main method not found` error at runtime.

Q21. What happens if main is not static?

Runtime error: `Main method is not static in class X.`

Q22. What if the class has no main method?

Runtime error: `Main method not found in class X, please define the main method as:`
`public static void main(String[] args).`

Q23. Can we change the name 'args' in String[] args?

Yes. `args` is just a parameter name. You can use any name. JVM only cares about the type (`String[]`).

Q24. Is Java case-sensitive?

Yes. `Main`, `main`, and `MAIN` are three different identifiers. JVM only recognizes `main` (lowercase) as entry point.

Section 5: First Program & Execution Flow

Q25. Write a basic Hello World program.

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Q26. What is the step-by-step execution flow?

1. Write `Hello.java` 2. `javac Hello.java` → compiles to `Hello.class` (bytecode) 3. `java Hello` → JVM loads the class via `ClassLoader` 4. JVM verifies bytecode, allocates memory 5. JVM calls `public static void main(String[] args)` 6. Program runs; JVM performs GC and exits

Q27. Difference between print, println, and printf?

`System.out.print("Hi")` — prints without newline `System.out.println("Hi")` — prints with newline `System.out.printf("%.2f", 3.14)` — formatted output

Q28. Can one .java file have multiple classes?

Yes, but only one can be `public`, and the filename must match that public class. Each class gets its own `.class` file.

Q29. What does the static block do and when does it run?

Runs **once when the class is loaded** by JVM, before `main` is called.

```
public class Demo {  
    static { System.out.println("Static block runs first"); }  
    public static void main(String[] args) {  
        System.out.println("main runs second");  
    }  
}
```

Topic 2 — Data Types & Variables

Section 1: Primitive Data Types

Q1. How many primitive data types are there in Java?

8 primitive types: byte, short, int, long, float, double, char, boolean

Type	Size	Default	Range / Notes
byte	1 byte	0	-128 to 127
short	2 bytes	0	-32,768 to 32,767
int	4 bytes	0	-2 ³¹ to 2 ³¹ -1
long	8 bytes	0L	-2 ⁶³ to 2 ⁶³ -1
float	4 bytes	0.0f	~7 decimal digits
double	8 bytes	0.0d	~15 decimal digits
char	2 bytes	\u0000	0 to 65,535 (Unicode)
boolean	JVM-dep.	false	true or false only

Q2. Why is char 2 bytes in Java but 1 byte in C?

Java uses **Unicode (UTF-16)** encoding (2 bytes). C uses ASCII (1 byte).

Q3. What is the default value of local variables?

No default value. Using a local variable before initialization causes a compile-time error. Only instance and static variables get defaults.

Section 2: Variable Types & Scope

Q4. What are the types of variables in Java?

Local — inside a method/block; no default value **Instance** — inside class, outside methods; has default; one per object **Static** — declared with `static`; shared across all objects

```
class Demo {
    static int count = 0;    // static variable
    String name;            // instance variable
    void show() { int x = 5; } // local variable
}
```

Section 3: Type Casting

Q5. What is widening (implicit) casting?

Auto-converting a smaller type to a larger type. No data loss. Order: `byte` → `short` → `int` → `long` → `float` → `double`

Q6. What is narrowing (explicit) casting?

Manually converting a larger type to a smaller type. May cause data loss.

```
double d = 9.99;
int i = (int) d;    // explicit narrowing
System.out.println(i); // 9 (decimal part lost!)
```

Q7. Can we cast boolean to int?

No. `boolean` is completely incompatible with numeric types in Java.

Q8. What is the result of (byte) 130?

-126. Byte range is -128 to 127. 130 overflows: $130 - 256 = -126$.

Section 4: Wrapper Classes & Autoboxing

Q9. What are wrapper classes?

Primitive	Wrapper
int	Integer
long	Long
double	Double
float	Float
byte	Byte
short	Short
char	Character
boolean	Boolean

Q10. What is autoboxing and unboxing?

Autoboxing — automatic primitive → wrapper (Java 5+) **Unboxing** — automatic wrapper → primitive

```
Integer obj = 5;    // autoboxing
int x = obj;        // unboxing
```

Q11. What is Integer caching? Why does == behave differently?

Java caches `Integer` objects from **-128 to 127**. Values in this range return the same cached object (`==` returns `true`). Outside this range, new objects are created. Always use `.equals()`.

```
Integer a = 100, b = 100;
System.out.println(a == b);    // true (cached)
Integer x = 200, y = 200;
System.out.println(x == y);    // false (new objects)
System.out.println(x.equals(y)); // true
```

Section 5: final, var, and Constants

Q12. What does final do on a variable?

Can be assigned only **once**. For objects, the reference is fixed but contents can change.

Q13. What is var in Java? (Java 10+)

`var` is **local variable type inference**. Compiler infers the type from the right-hand side. Works only for local variables.

```
var name = "Java";    // inferred as String
var list = new ArrayList<>(); // inferred as ArrayList
```

Section 6: String as a Data Type

Q14. Is String a primitive type?

No. `String` is a class in `java.lang`. It has special support (string literals, `+` operator, string pool).

Q15. What is the String Pool?

A special memory area in the heap where string literals are stored. JVM reuses existing pool objects for the same literal.

Q16. Why is String immutable?

Security (used in class loading), thread safety, string pool efficiency, and hashCode caching.

Q17. Difference between String, StringBuilder, and StringBuffer?

String — immutable; new object on every change **StringBuilder** — mutable; not synchronized; faster; not thread-safe **StringBuffer** — mutable; synchronized; thread-safe; slightly slower

Section 7: Tricky Questions

Q18. What is the result of (int) 130 stored in byte?

-126 due to overflow truncation.

Q19. What does `0.1 + 0.2 == 0.3` evaluate to?

false — IEEE 754 floating-point precision: `0.1 + 0.2 = 0.30000000000000004`. Use `BigDecimal` for exact decimal arithmetic.

Q20. What is NaN and Infinity in Java?

`NaN = 0.0 / 0.0`. Note: `NaN != NaN` is `true`! Use `Double.isNaN(x)`. `Infinity = 1.0 / 0.0`. Integer division by zero throws `ArithmeticException` but float/double gives `Infinity`.

Topic 3 — Operators in Java

Section 1: Arithmetic Operators

Operator	Meaning	Example	Result
+	Addition	5+3	8
-	Subtraction	5-3	2
*	Multiplication	5*3	15
/	Division	5/2	2
%	Modulus	5%2	1

Q1. What does + do with Strings?

+ acts as **string concatenation** when either operand is a String. Evaluation is left to right.

```
System.out.println(5 + 5 + "Hello"); // 10Hello
System.out.println("Hello" + 5 + 5); // Hello55
```

Section 2: Logical Operators & Short-Circuit

Q2. What is short-circuit evaluation?

&& — if left side is **false**, right side is NOT evaluated || — if left side is **true**, right side is NOT evaluated

Q3. Difference between & and &&?

&& — logical AND with **short-circuit** (stops early) & — bitwise AND, evaluates **both sides always** — dangerous with null checks!

Section 3: Bitwise Operators

Operator	Name	Example	Result
&	AND	5 & 3 (0101 & 0011)	0001 = 1
	OR	5 3 (0101 0011)	0111 = 7
^	XOR	5 ^ 3 (0101 ^ 0011)	0110 = 6
~	NOT	~5	-(5+1) = -6

Q4. How to check even/odd using bitwise?

```
if ((n & 1) == 0) // even
if ((n & 1) == 1) // odd
```

Section 4: Shift Operators

Q5. What are shift operators?

<< Left shift — multiply by 2^n >> Right shift (signed) — divide by 2^n , preserves sign bit >>> Unsigned right shift — fills 0 always

```
5 << 1 = 10    (5 * 2)
20 >> 2 = 5     (20 / 4)
-8 >> 1 = -4    (sign preserved)
-8 >>> 1 = large positive number (sign bit becomes 0)
```

Section 5: Unary & Ternary Operators

Q6. Difference between pre-increment and post-increment?

`++a` — increment first, then use `a++` — use first, then increment

```
int a = 5;
System.out.println(++a); // 6
System.out.println(a++); // 6 (then a becomes 7)
System.out.println(a);   // 7
```

Q7. What is the ternary operator?

Syntax: `condition ? valueIfTrue : valueIfFalse`

```
int max = (a > b) ? a : b;
```

Q8. Does compound assignment do implicit casting?

Yes! `b += 1` works even for `byte b`, but `b = b + 1` is a compile error (result is `int`).

Section 6: Operator Precedence

Priority	Operators
Highest	<code>++ --</code> (post), <code>()</code> , <code>[]</code>
	<code>++ --</code> (pre), <code>!</code> , <code>~</code> , unary <code>-</code>
	<code>*</code> , <code>/</code> , <code>%</code>
	<code>+</code> , <code>-</code>
	<code><<</code> , <code>>></code> , <code>>>></code>
	<code><</code> , <code>></code> , <code><=</code> , <code>>=</code> , <code>instanceof</code>
	<code>==</code> , <code>!=</code>
	<code>&</code>
	<code>^</code>
	<code> </code>
	<code>&&</code>
	<code> </code>
	<code>?:</code> (ternary)
Lowest	<code>=</code> , <code>+=</code> , <code>-=</code> etc.

Topic 4 — Control Statements in Java

Section 1: if, if-else, if-else-if

Q1. Can we write if without curly braces {}?

Yes, but only the **immediately next single statement** belongs to the `if`. Always use braces to avoid bugs.

Q2. Can we use if with non-boolean conditions like in C?

No. Java strictly requires a `boolean` expression inside `if`. `if(1)` is a compile error.

Q3. What is the dangling else problem?

`else` always pairs with the **nearest preceding** `if` without braces.

Section 2: switch Statement

Q4. What data types can be used in switch?

byte, short, int, char (from start) · String (Java 7+) · enum (Java 5+) · Byte, Short, Integer, Character wrapper classes **Cannot use:** long, float, double, boolean

Q5. What is fall-through in switch?

If `break` is omitted, execution falls through to the next case regardless of its value.

```
int x = 1;
switch(x) {
    case 1: System.out.println("One");    // prints
    case 2: System.out.println("Two");    // also prints!
    case 3: System.out.println("Three"); break;
}
// Output: One Two Three
```

Q6. What is the enhanced switch expression? (Java 14+)

```
String name = switch (day) {
    case 1 -> "Monday";
    case 2 -> "Tuesday";
    default -> "Other";
};
```

Section 3: Loops — for, while, do-while

Q7. What is the scope of the variable declared in for initialization?

Limited to the `for` loop block only. Cannot be accessed after the loop.

Q8. Difference between while and do-while?

Feature	while	do-while
Condition check	before body	after body
Min executions	0 (may never run)	1 (always runs once)

Q9. Can enhanced for loop modify array elements?

For primitives — **No.** Changing the loop variable does not affect the original array.

Section 4: break, continue, return

Q10. Difference between break and continue?

`break` — exits the loop entirely `continue` — skips current iteration only, loop continues

Q11. What is a labeled break?

Used to break out of an **outer loop** from inside a nested loop.

```
outer:
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        if (j == 1) break outer; // exits outer loop
    }
}
```

Q12. Is there a goto in Java?

`goto` is a **reserved keyword** in Java but has **no functionality**. Java uses labeled `break` and `continue` instead.

Q13. What is unreachable code?

Code after `return`, `break`, or `continue` that can never execute. Java gives a **compile error** for unreachable code.

Topic 5 — Arrays in Java

Section 1: Array Basics

Q1. What is an array in Java?

A **fixed-size**, ordered collection of elements of the **same data type** stored in contiguous memory. Arrays are objects stored on the heap.

Q2. Default values of array elements?

Type	Default Value
int, byte, short, long	0
float, double	0.0
char	\u0000 (null char)
boolean	false
Object (String etc.)	null

Q3. How do you initialize an array?

```
int[] arr = new int[3];           // default values
arr[0] = 10; arr[1] = 20;
int[] arr2 = {10, 20, 30};       // array literal
int[] arr3 = new int[]{10, 20};  // new keyword with values
```

Q4. What is ArrayIndexOutOfBoundsException?

Thrown when accessing an invalid index — negative or $\geq$ array length.

Section 2: Array Copying

Q5. How do you copy an array?

```
Arrays.copyOf(original, original.length)
Arrays.copyOfRange(original, 1, 4)
System.arraycopy(src, 0, dest, 0, length)
original.clone()
```

Q6. What is the difference between shallow and deep copy?

Shallow copy — copies references; both arrays point to same objects. **Deep copy** — copies actual objects; changes to one don't affect the other. For primitive arrays, all copy methods produce a true independent (deep) copy.

Section 3: 2D and Jagged Arrays

Q7. How do you declare and iterate a 2D array?

```
int[][] matrix = {{1,2,3},{4,5,6},{7,8,9}};
for (int i = 0; i < matrix.length; i++)
    for (int j = 0; j < matrix[i].length; j++)
        System.out.print(matrix[i][j] + " ");
```

Q8. What is a jagged array?

An array of arrays where each row can have a **different number of columns**.

```
int[][] jagged = new int[3][];
jagged[0] = new int[2]; // 2 elements
jagged[1] = new int[4]; // 4 elements
```

Section 4: Tricky Questions

Q9. What happens when you assign one array to another?

Both variables point to the **same array object**. Changes through one affect the other.

```
int[] a = {1,2,3};
int[] b = a;          // same object!
b[0] = 99;
System.out.println(a[0]); // 99
```

Q10. What is the difference between length and length()?

`array.length` — property of arrays (no parentheses) `string.length()` — method of String class (with parentheses)

Q11. What is ArrayStoreException?

Thrown when storing an incompatible type into an object array.

```
Object[] arr = new String[3];
arr[0] = "Hello"; // OK
arr[1] = 42;      // ArrayStoreException at runtime!
```

Topic 6 — OOP: Classes, Objects & Constructors

Section 1: OOP Concepts Overview

Q1. What are the 4 pillars of OOP?

Encapsulation — wrapping data and methods; hiding internal details **Inheritance** — one class acquiring properties of another **Polymorphism** — one entity behaving differently in different situations **Abstraction** — hiding implementation, showing only functionality

Q2. Difference between class and object?

Class	Object
Blueprint / template	Real-world instance
Declared once	Created many times
No memory at declaration	Memory allocated when created
Example: Car (design)	Example: myCar (actual car)

Section 2: Constructors

Q3. What are the types of constructors?

Default constructor — no args; provided by compiler if none defined **No-arg constructor** — no args; explicitly defined **Parameterized constructor** — takes arguments **Copy constructor** — creates a copy from another object

Q4. What happens if you define a parameterized constructor but not a no-arg?

Compiler **no longer provides** the default constructor. Calling `new Car()` causes a compile error.

Q5. Can a constructor be private?

Yes. Prevents external object creation. Used in **Singleton pattern** and utility classes.

Q6. Can a constructor be final, static, or abstract?

No. None of these modifiers are valid for constructors.

Q7. Can constructors be overloaded?

Yes. Multiple constructors with different parameter lists — very common.

Q8. Can constructors be inherited?

No. Constructors are not inherited. Child uses `super()` to call parent constructor.

Section 3: this Keyword

Q9. What are the uses of this?

1. Refer to current instance variable (resolve shadowing) 2. Call current class method 3. Call current class constructor — `this()` 4. Pass current object as argument 5. Return current object from method

Q10. What is this() constructor call?

Calls **another constructor of the same class**. Must be the **first statement**.

Q11. Can this() and super() both be first statement?

No. Only one can be the first statement in a constructor.

Section 4: Object Creation & GC

Q12. What is the order of execution when an object is created?

1. Static blocks (only once, when class is first loaded) 2. Instance initializer blocks (every object creation) 3. Constructor body

Q13. When is an object eligible for garbage collection?

When no live thread can reach it through any chain of references — set to `null` or goes out of scope.

Q14. What is the contract between `equals()` and `hashCode()`?

If two objects are equal via `equals()`, they **must** have the same `hashCode()`. Always override both together.

Topic 7 — Inheritance in Java

Section 1: Types of Inheritance

Type	Description	Supported?
Single	One child, one parent	Yes
Multilevel	A extends B, B extends C	Yes
Hierarchical	Multiple children, one parent	Yes
Multiple	One child, multiple parent classes	No (use interfaces)
Hybrid	Combination of above	No (with classes)

Q1. Why doesn't Java support multiple inheritance with classes?

To avoid the **Diamond Problem** — ambiguity when two parent classes have a method with the same name. Solved through interfaces.

Section 2: super Keyword

Q2. What is the super keyword?

Refers to the **immediate parent class**. Used to access parent fields, call parent methods, and call parent constructor via `super()`.

Q3. Is super() automatically called if not written?

Yes. Compiler inserts `super()` (no-arg) automatically. If parent has no no-arg constructor, compile error.

Section 3: Method Overriding

Q4. What are the rules for method overriding?

Same method name · Same parameters · Same or covariant return type · Access modifier cannot be more restrictive · Cannot override `static`, `final`, or `private` methods

Q5. Can we override a static method?

No. Static methods use **method hiding** not overriding. Resolved at compile time based on reference type.

Q6. What is covariant return type?

Since Java 5, an overriding method can return a **subtype** of the parent method's return type.

Q7. Difference between overloading and overriding?

Feature	Overloading	Overriding
Class	Same class	Parent and child
Parameters	Must differ	Must be same
Binding	Compile time	Runtime
Polymorphism	Compile-time	Runtime

Section 4: Constructors in Inheritance

Q8. What is the order of constructor execution in inheritance?

Parent constructor always executes **before** child constructor.

```
// new C() outputs:  
// A constructor
```

```
// B constructor  
// C constructor
```

Q9. Can a child class access private members of parent?

No. Private members are not inherited. Use `protected` or public getters/setters.

Topic 8 — Polymorphism in Java

Section 1: Types of Polymorphism

Type	Also Called	Resolved At	Achieved By
Compile-time	Static polymorphism	Compile time	Method overloading
Runtime	Dynamic polymorphism	Runtime	Method overriding + upcasting

Section 2: Runtime Polymorphism

Q1. What is upcasting?

Assigning a **child class object** to a parent class reference. Done implicitly. `Animal a = new Dog();`

Q2. What is downcasting?

Converting a parent reference back to a child type. Done **explicitly**. May throw `ClassCastException`.

```
Animal a = new Dog(); // upcasting
Dog d = (Dog) a;      // downcasting - safe
Animal a2 = new Animal();
Dog d2 = (Dog) a2;    // ClassCastException at runtime!
```

Q3. How do you safely downcast?

Use `instanceof` before downcasting.

Q4. Do fields support runtime polymorphism?

No. Fields are resolved at **compile time** based on reference type. Only methods are dynamically dispatched.

Q5. Do static methods support runtime polymorphism?

No. Static methods use **method hiding** — resolved at compile time based on reference type.

Section 3: Tricky Questions

Q6. What is the output?

```
class Parent { void show() { System.out.println("Parent"); }
               void display() { show(); } }
class Child extends Parent { void show() { System.out.println("Child"); } }
Parent p = new Child();
p.display(); // Child show
```

Child — `display()` is not overridden so Parent's runs. Inside, `show()` is called on the actual Child object → runtime polymorphism.

Q7. What is early binding vs late binding?

Early binding (static) — resolved at compile time: `static`, `final`, `private`, overloaded methods.

Late binding (dynamic) — resolved at runtime: overridden instance methods.

Topic 9 — Abstraction: Abstract Classes & Interfaces

Section 1: Abstract Classes

Q1. Can an abstract class have a constructor?

Yes. Called when subclass object is created via `super()`.

Q2. Can abstract method be static, final, or private?

No to all three. Static can't be overridden. Final can't be overridden. Private not visible to subclasses. All contradict abstract.

Q3. Can a class with no abstract methods be declared abstract?

Yes. Prevents direct instantiation even without abstract methods.

Q4. What if a subclass doesn't implement all abstract methods?

The subclass must also be declared `abstract`, otherwise compile error.

Section 2: Interfaces

Q5. What are the default modifiers of interface members?

Member	Default Modifier
Methods (before Java 8)	public abstract
Variables	public static final
Methods (Java 8 default)	public
Methods (Java 9 private)	private

Q6. Can a class implement multiple interfaces?

Yes. This is how Java achieves multiple inheritance.

Q7. What happens when two interfaces have the same default method?

Compile error — ambiguity. The implementing class **must override** the method. Can call specific version with `A.super.method()`.

Section 3: Abstract Class vs Interface

Feature	Abstract Class	Interface
Instantiation	No	No
Methods	abstract + concrete	abstract + default + static
Variables	all types	public static final only
Constructor	Yes	No
Multiple inheritance	No	Yes
When to use	Shared code + partial abstraction	Pure contract / multiple inheritance

Section 4: Functional & Marker Interfaces

Q8. What is a functional interface?

An interface with **exactly one abstract method**. Used with lambda expressions. Annotated with `@FunctionalInterface`.

Q9. What is a marker interface?

An interface with **no methods or fields** — just marks a class for special treatment. Examples: `Serializable`, `Cloneable`, `Remote`.

Q10. Can an interface be final?

No. A final interface cannot be extended or implemented — makes it useless. Compile error.

Topic 10 — Encapsulation, Packages & Access Modifiers

Section 1: Encapsulation

Q1. How do you achieve encapsulation?

Declare all fields as **private**. Provide **public getter** methods to read and **public setter** methods to modify with validation.

```
class Employee {  
    private String name;  
    private int salary;  
    public String getName() { return name; }  
    public void setName(String name) { this.name = name; }  
    public void setSalary(int salary) {  
        if (salary > 0) this.salary = salary; // validation  
    }  
}
```

Q2. What is a POJO?

Plain Old Java Object — private fields, public getters/setters, no-arg constructor, no special framework dependency.

Q3. What is a Java Bean?

A POJO that also implements `Serializable`. Used heavily in Spring, Hibernate, JSP.

Section 2: Access Modifiers

Modifier	Same Class	Same Package	Subclass (diff pkg)	Other Package
private	Yes	No	No	No
default	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

Q4. Can a top-level class be private or protected?

No. Top-level class can only be `public` or default. Inner classes can use all four modifiers.

Q5. Can we reduce visibility of overriding method?

No. Overriding method cannot have a **more restrictive** access modifier. Can widen (e.g., protected → public).

Section 3: Packages

Q6. What are built-in packages?

Package	Contains
java.lang	String, Math, Object, System, Thread (auto-imported)
java.util	ArrayList, HashMap, Scanner, Date, Collections
java.io	File, InputStream, OutputStream, BufferedReader
java.net	Socket, URL, HttpURLConnection
java.sql	Connection, Statement, ResultSet

Q7. What is static import?

Allows importing static members directly without the class name prefix.

```
import static java.lang.Math.PI;  
System.out.println(PI); // instead of Math.PI
```

Q8. What is the final keyword on variables, methods, and classes?

Applied to	Effect
Variable	Value cannot be changed (constant)
Method	Cannot be overridden
Class	Cannot be subclassed
Parameter	Parameter cannot be changed inside method

Topic 11 — Exception Handling in Java

Section 1: Exception Hierarchy

Q1. Difference between Error and Exception?

Feature	Error	Exception
Package	java.lang.Error	java.lang.Exception
Cause	JVM-level serious problems	Program-level problems
Recoverable?	No	Yes (mostly)
Examples	OutOfMemoryError, StackOverflowError	NullPointerException, IOException

Q2. Difference between checked and unchecked exceptions?

Feature	Checked	Unchecked
Also called	Compile-time exceptions	Runtime exceptions
Detected at	Compile time	Runtime
Must handle?	Yes — compile error if not	No — optional
Extends	Exception (not RuntimeException)	RuntimeException
Examples	IOException, SQLException	NullPointerException, ArithmeticException

Section 2: try-catch-finally

Q3. What is the order of catch blocks?

More **specific** exceptions must come before more **general** ones. Parent exception caught first → child catch block unreachable → compile error.

Q4. When does finally NOT execute?

When `System.exit()` is called, when JVM crashes, or when the thread is killed.

Q5. What is multi-catch? (Java 7)

```
catch (IOException | SQLException e) {  
    System.out.println("IO or SQL error: " + e.getMessage());  
}
```

Q6. What is the output?

```
try { System.out.println("try"); return; }  
finally { System.out.println("finally"); }
```

try then finally — **finally** runs even when **return** is in try block.

Section 3: throw and throws

Q7. Difference between throw and throws?

Feature	throw	throws
Purpose	Actually throws an exception	Declares possible exceptions
Location	Inside method body	Method signature
Count	One exception at a time	Multiple allowed

Section 4: Custom Exceptions & try-with-resources

Q8. How do you create a custom exception?

```
class InsufficientFundsException extends Exception {  
    InsufficientFundsException(double amount) {  
        super("Insufficient funds. Need: " + amount);  
    }  
}
```

Q9. What is try-with-resources? (Java 7)

Automatically **closes resources** after the block. Resource must implement `AutoCloseable`.

```
try (FileReader fr = new FileReader("file.txt");  
    BufferedReader br = new BufferedReader(fr)) {  
    System.out.println(br.readLine());  
} // fr and br automatically closed
```

Q10. Difference between final, finally, and finalize?

Keyword	Type	Purpose
final	Keyword	Constant / no override / no extend
finally	Block	Always-execute cleanup block
finalize	Method	Called before GC (deprecated Java 9, removed Java 18)

Q11. Common exceptions and their causes?

Exception	Cause
NullPointerException	Method called on null reference
ArrayIndexOutOfBoundsException	Invalid array index
ClassCastException	Invalid type cast
ArithmeticException	Math error e.g. divide by zero
NumberFormatException	Integer.parseInt("abc")
StackOverflowError	Infinite recursion
ConcurrentModificationException	Modifying collection while iterating

Topic 12 — Strings in Java (Deep Dive)

Section 1: String Basics & Immutability

Q1. How many ways can you create a String?

```
String s1 = "Hello";           // literal (pool)
String s2 = new String("Hello"); // heap object
char[] ch = {'H','e','l','l','o'};
String s3 = new String(ch);    // from char array
```

Q2. Why is String immutable?

Security (class loading, network), **Thread safety** (immutable = inherently safe), **String Pool** efficiency (pool only works if strings can't change), **HashCode caching** (never changes).

Q3. What is string interning?

Adding a string to the String Pool manually using `intern()`. Returns the pool reference.

```
String s1 = new String("Hello"); // heap
String s2 = s1.intern();         // pool reference
String s3 = "Hello";            // pool
System.out.println(s2 == s3);   // true
```

Section 2: Important String Methods

```
String s = "Hello World";
s.length()           // 11
s.charAt(0)          // 'H'
s.indexOf('o')        // 4
s.substring(6)        // "World"
s.substring(0, 5)     // "Hello"
s.toLowerCase()      // "hello world"
s.toUpperCase()       // "HELLO WORLD"
s.trim()              // removes leading/trailing spaces
s.strip()             // Unicode-aware trim (Java 11)
s.replace("Hello", "Hi") // "Hi World"
s.contains("World")    // true
s.startsWith("Hello") // true
s.split(" ")          // ["Hello", "World"]
s.equals("Hello World") // true
s.equalsIgnoreCase("hello world") // true
s.compareTo("Hello")   // 0 if equal
"ha".repeat(3)         // "hahaha" (Java 11)
```

Section 3: String Comparison Tricky Questions

Q4. What is the output?

```
String s1 = "Java";
String s2 = "Jav" + "a"; // compile-time constant → pool
String s3 = "Jav";
String s4 = s3 + "a";    // runtime concat → heap
System.out.println(s1 == s2); // true
System.out.println(s1 == s4); // false
```

Section 4: StringBuilder vs StringBuffer

Q5. Why use StringBuilder over String for concatenation?

String `+` in a loop creates many intermediate objects ($O(n^2)$). **StringBuilder** uses a **mutable char array** — $O(n)$. Use **StringBuilder** for any string building in loops.

Feature	StringBuilder	StringBuffer
---------	---------------	--------------

Thread-safe	No	Yes (synchronized)
Performance	Faster	Slower
Introduced	Java 5	Java 1.0
Use case	Single-threaded	Multi-threaded

Q6. What are important StringBuilder methods?

```
sb.append(" World") // add at end
sb.insert(5, " Hi") // insert at index
sb.delete(5, 10)    // remove chars 5-9
sb.reverse()        // reverse
sb.replace(0,5,"Hi")// replace range
sb.toString()       // convert to String
```

Topic 13 — Collections Framework in Java

Section 1: Hierarchy Overview

Q1. Difference between Collection and Collections?

Feature	Collection	Collections
Type	Interface	Utility class
Purpose	Root interface for data structures	Static utility methods
Example	List, Set, Queue	Collections.sort(), shuffle()

Section 2: List — ArrayList & LinkedList

Q2. What is the default capacity and growth of ArrayList?

Default capacity: **10**. Grows to approximately **1.5x** when full (old capacity + old/2).

Q3. Difference between ArrayList and LinkedList?

Feature	ArrayList	LinkedList
Internal structure	Dynamic array	Doubly linked list
Random access get(i)	O(1) — fast	O(n) — slow
Insertion at end	O(1) amortized	O(1)
Insertion at middle	O(n) — slow	O(1) once at position
Best for	Read-heavy	Insert/delete-heavy

Section 3: Set — HashSet, LinkedHashSet, TreeSet

Feature	HashSet	LinkedHashSet	TreeSet
Order	No order	Insertion order	Sorted order
null	One null	One null	No null
Performance	O(1)	O(1) slightly slower	O(log n)
Internal	HashMap	LinkedHashMap	Red-Black Tree

Section 4: Map — HashMap, LinkedHashMap, TreeMap

Q4. How does HashMap work internally?

Uses an array of **buckets** (default 16). Key's `hashCode()` determines bucket. `equals()` resolves collisions. Java 8+: bucket converts to **Red-Black tree** when size > 8. **Load factor** 0.75 triggers rehashing.

Q5. Difference between HashMap, LinkedHashMap, TreeMap?

Feature	HashMap	LinkedHashMap	TreeMap
Order	No order	Insertion order	Sorted by key
null key	One null	One null	No null
Performance	O(1)	O(1) slower	O(log n)

Q6. Difference between HashMap and Hashtable?

Feature	HashMap	Hashtable
---------	---------	-----------

Thread-safe	No	Yes
null key	Allowed	Not allowed
Performance	Faster	Slower
Legacy	No	Yes (Java 1.0)

Section 5: Queue, Deque, Comparable, Comparator

Q7. Difference between poll() and remove() in Queue?

`poll()` returns `null` if empty. `remove()` throws `NoSuchElementException` if empty.

Q8. Difference between Comparable and Comparator?

Feature	Comparable	Comparator
Package	java.lang	java.util
Method	compareTo(T o)	compare(T o1, T o2)
Modifies class	Yes	No
Sort orders	One (natural)	Multiple custom

Q9. What is ConcurrentModificationException?

Thrown when you modify a collection while iterating. Use `Iterator.remove()` or `removeIf()` for safe removal.

Q10. Difference between fail-fast and fail-safe iterators?

Feature	Fail-fast	Fail-safe
Throws	ConcurrentModificationException	No exception
Works on	Original collection	Copy of collection
Examples	ArrayList, HashMap iterators	CopyOnWriteArrayList, ConcurrentHashMap

Topic 14 — Generics in Java

Section 1: Generic Classes & Methods

Q1. How do you create a generic class?

```
class Box<T> {  
    private T value;  
    Box(T value) { this.value = value; }  
    T getValue() { return value; }  
}  
Box<Integer> intBox = new Box<>(42);  
Box<String> strBox = new Box<>("Hello");
```

Q2. Can you use primitives as type parameters?

No. Generics only work with reference types. Use wrapper classes: `Box<Integer>` not `Box<int>`.

Section 2: Wildcards & Bounded Types

Q3. What is an upper bounded wildcard?

`<? extends Type>` — accepts `Type` and subtypes. Used for **reading** from a collection.

Q4. What is a lower bounded wildcard?

`<? super Type>` — accepts `Type` and supertypes. Used for **writing/adding** to a collection.

Q5. What is the PECS principle?

Producer Extends, Consumer Super Use `<? extends T>` when reading (producing data). Use `<? super T>` when writing (consuming data).

Q6. Difference between `List<?>` and `List<Object>`?

`List<Object>` only accepts `ArrayList<Object>` — no polymorphism. `List<?>` accepts `ArrayList<AnythingAtAll>` — truly generic. But with `List<?>` you cannot add elements (except null).

Section 3: Type Erasure

Q7. What is type erasure?

At compile time, the compiler removes all generic type info and replaces with bounds (or `Object`). At runtime, generic type info does not exist.

Q8. Limitations due to type erasure?

```
T obj = new T();           // Compile error!  
T[] arr = new T[10];       // Compile error!  
if (obj instanceof T) {}   // Compile error!  
void method(List<String> l) {} // same as:  
void method(List<Integer> l){} // Compile error - same after erasure!
```

Q9. Can static methods use generic class type parameters?

No. Static context doesn't belong to any instance. Static methods must declare their own type parameter.

```
class Box<T> {  
    static T getDefault() { return null; } // Compile error!  
    static <E> E getDefault(E val) { return val; } // OK  
}
```

Topic 15 — Java 8 Features

Section 1: Lambda Expressions

Q1. What is a lambda expression?

An **anonymous function** — code without a name, passed like a value. Concise way to implement functional interfaces.

```
// No parameter
() -> System.out.println("Hello")
// One parameter
x -> x * x
// Multiple parameters
(a, b) -> a + b
// Multiple statements
(a, b) -> { int sum = a + b; return sum; }
```

Q2. What is an effectively final variable?

A local variable never reassigned after initialization. Lambdas can use effectively final locals. Reassigning after use in lambda → compile error.

Section 2: Functional Interfaces

Interface	Method	Description
Predicate<T>	test(T)	Takes T, returns boolean
Function<T,R>	apply(T)	Takes T, returns R
Consumer<T>	accept(T)	Takes T, returns nothing
Supplier<T>	get()	No input, returns T
BiFunction<T,U,R>	apply(T,U)	Takes T and U, returns R
UnaryOperator<T>	apply(T)	Takes T, returns T
BinaryOperator<T>	apply(T,T)	Takes two T, returns T

Section 3: Method References

Q3. What are the 4 types of method references?

```
// Static method reference
Function<String, Integer> parser = Integer::parseInt;
// Specific instance method reference
Supplier<String> upper = str::toUpperCase;
// Arbitrary instance method reference
Function<String, String> toUpper = String::toUpperCase;
// Constructor reference
Supplier<ArrayList> factory = ArrayList::new;
```

Section 4: Stream API

Q4. What are intermediate operations?

Transform a stream into another stream. **Lazy** — not executed until terminal operation is called.

```
list.stream()
    .filter(n -> n % 2 == 0) // keep evens
    .map(n -> n * n)         // square them
    .distinct()              // remove duplicates
    .sorted()                // sort
    .limit(5)                // take first 5
    .collect(Collectors.toList()); // terminal op
```

Q5. What are terminal operations?

`forEach` · `collect` · `count` · `reduce` · `min` · `max` · `findFirst` · `findAny` · `anyMatch` · `allMatch` · `noneMatch` · `toArray` · `sum` (numeric streams)

Q6. Difference between map and flatMap?

`map` — one-to-one transformation (returns `Stream<Stream<T>>` for nested) `flatMap` — one-to-many, **flattens** nested streams into one `Stream<T>`

Q7. Is streams lazy? Prove it.

```
numbers.stream()
    .filter(n -> { System.out.println("filter: "+n); return n>2; })
    .map(n -> { System.out.println("map: "+n); return n*n; })
    .findFirst();
// Only processes elements until first match found!
```

Q8. Can a stream be reused?

No. After a terminal operation, the stream is closed. Reusing throws `IllegalStateException`.

Section 5: Optional Class**Q9. What are important Optional methods?**

```
Optional<String> opt = Optional.of("Hello");
opt.isPresent()           // true
opt.get()                 // "Hello"
opt.orElse("Default")    // value or default
opt.orElseGet(() -> computeDefault()) // lazy default
opt.orElseThrow()         // throw if empty
opt.ifPresent(s -> print(s)) // action if present
opt.map(s -> s.toUpperCase()) // transform
opt.filter(s -> s.length() > 3) // filter
```

Q10. Difference between orElse() and orElseGet()?

`orElse(value)` — default is **always evaluated** even if Optional has value. `orElseGet(supplier)` — supplier only called if Optional is **empty** (lazy — preferred for expensive operations).

Section 6: New Date/Time API

Class	Date	Time	Timezone
<code>LocalDate</code>	Yes	No	No
<code>LocalTime</code>	No	Yes	No
<code>LocalDateTime</code>	Yes	Yes	No
<code>ZonedDateTime</code>	Yes	Yes	Yes

```
LocalDate today = LocalDate.now();
LocalDate date = LocalDate.of(2024, 1, 15);
date.plusDays(10); date.minusMonths(1);
DateTimeFormatter fmt = DateTimeFormatter.ofPattern("dd/MM/yyyy");
String formatted = today.format(fmt); // "15/01/2024"
```

Topic 16 — Multithreading in Java

Section 1: Creating Threads

Q1. Which is better — extending Thread or implementing Runnable?

Implementing Runnable is preferred — Java allows only single inheritance, so extending Thread prevents extending another class. Runnable is also more flexible with Executor framework.

Q2. Difference between start() and run()?

start()	run()
Creates a new thread	No new thread created
JVM calls run() in new thread	Runs in current thread
Can only be called once	Can be called multiple times
Asynchronous	Synchronous

Q3. Difference between Runnable and Callable?

Feature	Runnable	Callable
Return type	void	Generic V
Exception	Cannot throw checked	Can throw checked
Method	run()	call()
Result	Not possible	Via Future.get()

Section 2: Thread Lifecycle

State	Description
NEW	Thread created, start() not called
RUNNABLE	start() called — ready or running
BLOCKED	Waiting to acquire a monitor lock
WAITING	Waiting indefinitely (wait(), join())
TIMED_WAITING	Waiting for specified time (sleep(), wait(ms))
TERMINATED	Thread finished execution

Section 3: Synchronization

Q4. What is a race condition?

Two or more threads access shared data simultaneously and outcome depends on execution order → inconsistent results.

Q5. Difference between synchronized and volatile?

Feature	synchronized	volatile
Atomicity	Yes	No
Visibility	Yes	Yes
Blocking	Yes (threads block)	No
Use case	Compound operations	Single read/write flags

Q6. What is a deadlock?

Two threads are **waiting for each other** to release locks — neither can proceed. Prevent by: lock ordering, lock timeout (`tryLock()`), avoiding nested locks.

Q7. Difference between wait() and sleep()?

Feature	wait()	sleep()
Class	Object	Thread
Lock	Releases lock	Does NOT release lock
Called from	synchronized block only	Anywhere
Woken by	notify()/notifyAll()	Timeout or interrupt

Section 4: Executor Framework**Q8. What are the types of thread pools?**

```
Executors.newFixedThreadPool(5)           // fixed number
Executors.newCachedThreadPool()           // creates as needed
Executors.newSingleThreadExecutor()       // one thread, sequential
Executors.newScheduledThreadPool(3)       // delayed/periodic tasks
```

Q9. What is CompletableFuture? (Java 8)

An enhanced `Future` supporting **non-blocking** async programming with callbacks and chaining.

```
CompletableFuture.supplyAsync(() -> 42)
    .thenApply(r -> r * 2)           // transform
    .thenAccept(r -> print(r)); // consume - non-blocking
```

Q10. What is a daemon thread?

A background thread that supports user threads. JVM exits when only daemon threads remain. Set before calling `start()`: `t.setDaemon(true)`.

Topic 17 — Java I/O and File Handling

Section 1: Streams — Byte vs Character

Feature	Byte Streams	Character Streams
Data unit	8-bit byte	16-bit Unicode char
Base classes	InputStream, OutputStream	Reader, Writer
Use case	Images, audio, binary	Text files
Examples	FileInputStream, FileOutputStream	FileReader, FileWriter

Section 2: File Class & Operations

Q1. What does the File class do?

`java.io.File` represents a file or directory **path**. It manages metadata but does not read/write content.

```
File file = new File("test.txt");
file.exists(); file.isFile(); file.isDirectory();
file.getName(); file.getAbsolutePath(); file.length();
file.createNewFile(); file.delete(); file.mkdir();
file.listFiles(); // array of File in directory
```

Q2. How do you append to a file instead of overwriting?

Pass `true` as second argument: `new FileOutputStream("file.txt", true)` or `new FileWriter("file.txt", true)`

Section 3: Buffered I/O

Q3. What is BufferedReader and why use it?

Wraps a `Reader` and adds a **buffer** to minimize disk reads. Much faster than character-by-character reading.

```
try (BufferedReader br = new BufferedReader(new FileReader("test.txt"))) {
    String line;
    while ((line = br.readLine()) != null) {
        System.out.println(line);
    }
}
```

Section 4: Serialization

Q4. What is serialization?

Converting an object into a **byte stream** for storage or transmission. Class must implement `Serializable`. Deserialization is the reverse.

Q5. What is serialVersionUID?

A unique identifier for serializable classes. Verifies sender/receiver class compatibility during deserialization. Always declare explicitly: `private static final long serialVersionUID = 1L;`

Q6. What is the transient keyword?

Marks a field to be **excluded from serialization**. The field is not saved and will be null/default when deserialized. Used for passwords, cached values.

Q7. Can static fields be serialized?

No. Static fields belong to the class, not the object. They are not part of object state.

Section 5: Java NIO

Q8. What is the Files utility class?

```
Files.readAllLines(path)           // read all lines
Files.write(path, lines)           // write lines
Files.readString(path)             // Java 11
Files.writeString(path, content)   // Java 11
Files.copy(src, dest, REPLACE_EXISTING) // copy
Files.move(src, dest)              // move/rename
Files.delete(path)                 // delete
Files.walk(dir).filter(Files::isRegularFile) // walk tree
```

Topic 18 — Java Memory Management & Garbage Collection

Section 1: JVM Memory Areas

Memory Area	Description
Heap	Objects and arrays — shared by all threads
Stack	Method frames, local variables — one per thread
MetaSpace	Class metadata, static variables (Java 8+, replaced PermGen)
PC Register	Address of current instruction — one per thread
Native Method Stack	For native C/C++ method calls

Q1. Difference between Stack and Heap?

Feature	Stack	Heap
Stores	Primitives, references, frames	Objects, arrays
Speed	Faster	Slower
Thread	Own stack per thread	Shared by all
Error	StackOverflowError	OutOfMemoryError

Section 2: Garbage Collection

Q2. What are types of references?

Reference	Class	GC Behavior
Strong	Default	Never collected while strongly referenced
Soft	SoftReference<T>	Collected only when memory is low
Weak	WeakReference<T>	Collected at next GC cycle
Phantom	PhantomReference<T>	Collected after finalization

Q3. What is Minor GC vs Major GC vs Full GC?

Minor GC — Young Generation only (fast, frequent) **Major GC** — Old Generation (slower) **Full GC** — entire heap + MetaSpace (slowest, Stop-The-World pause)

Q4. Common GC algorithms?

Algorithm	Description
Serial GC	Single thread — small apps
Parallel GC	Multi-thread — default Java 8
G1 GC	Balanced — default Java 9+
ZGC	Ultra-low latency — Java 11+
Shenandoah	Low pause — Java 12+

Q5. Can Java have memory leaks?

Yes. Even with GC, objects kept unintentionally referenced never get collected. Common causes: static collections, unclosed streams, listeners not unregistered, ThreadLocal not removed, caches without eviction.

Topic 19 — Design Patterns in Java

Section 1: Creational Patterns

Q1. What is the Singleton pattern? Best implementation?

Ensures **only one instance** of a class. Best implementation: **Enum Singleton** — thread-safe, handles serialization, prevents reflection attacks.

```
enum Singleton { INSTANCE;
    public void doSomething() { System.out.println("Singleton!"); }
}
Singleton.INSTANCE.doSomething();
```

Q2. Thread-safe Singleton with double-checked locking?

```
class Singleton {
    private static volatile Singleton instance;
    private Singleton() { }
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

Q3. What is the Factory pattern?

Defines an interface for creating objects but lets subclasses decide which class to instantiate. Promotes loose coupling.

```
class ShapeFactory {
    static Shape getShape(String type) {
        switch(type) {
            case "circle": return new Circle();
            case "rectangle": return new Rectangle();
            default: throw new IllegalArgumentException();
        }
    }
}
```

Q4. What is the Builder pattern?

Constructs **complex objects step by step**. Separates construction from representation. Great for objects with many optional fields.

```
Person p = new Person.Builder()
    .name("Alice").age(25).email("a@email.com").build();
```

Section 2: Structural Patterns

Q5. What is the Adapter pattern?

Converts interface of a class into another interface clients expect. Allows **incompatible interfaces** to work together.

Q6. What is the Decorator pattern?

Adds new behavior to an object **dynamically** without changing its class. Wraps the original object.

Q7. What is the Facade pattern?

Provides a **simplified interface** to a complex subsystem. Hides complexity behind a simple API.

Section 3: Behavioral Patterns

Q8. What is the Observer pattern?

Defines a **one-to-many dependency** — when one object changes state, all dependents are notified automatically. Used in event systems, MVC.

Q9. What is the Strategy pattern?

Defines a family of algorithms, encapsulates each, makes them interchangeable. Allows changing algorithm at runtime.

Q10. What is the Template Method pattern?

Defines the **skeleton of an algorithm** in a base class, deferring specific steps to subclasses without changing the algorithm's structure.

Topic 20 — Miscellaneous: Enums, Nested Classes, Annotations & Modern Java

Section 1: Enums

Q1. What is an enum in Java?

A special class representing a **group of named constants**. Type-safe, can have fields, methods, and constructors.

```
enum Day { MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY }
Day d = Day.MONDAY;
d.ordinal(); // 0 (position)
d.name();    // "MONDAY"
Day.values(); // array of all values
```

Q2. Can enum have fields, constructors, and methods?

Yes. Enum is essentially a class and can have all of these.

Q3. Can enum implement an interface?

Yes. Each constant can provide its own implementation of interface methods.

Q4. Can enum extend a class?

No. Enums implicitly extend `java.lang.Enum` — multiple inheritance not supported.

Section 2: Nested & Inner Classes

Type	Static?	Access to Outer?	Description
Static nested class	Yes	Static members only	Like top-level class inside another
Inner class	No	All members including private	Needs outer class instance
Local class	No	Effectively final locals	Defined inside a method
Anonymous class	No	Effectively final locals	No name — one-time use

Q5. How do you create a non-static inner class object?

```
Outer outer = new Outer();
Outer.Inner inner = outer.new Inner(); // needs outer instance
```

Section 3: Annotations

Q6. What are built-in annotations?

Annotation	Purpose
@Override	Checks method overrides parent — compile error if not
@Deprecated	Marks element as outdated — compiler warning on use
@SuppressWarnings	Suppresses specific compiler warnings
@FunctionalInterface	Marks as functional interface — must have one abstract method
@SafeVarargs	Suppresses heap pollution warnings on varargs

Q7. What are meta-annotations?

Meta-annotation	Purpose
-----------------	---------

@Retention	How long retained: SOURCE, CLASS, RUNTIME
@Target	Where applied: METHOD, FIELD, CLASS etc.
@Inherited	Annotation inherited by subclasses
@Documented	Included in Javadoc
@Repeatable	Can be applied multiple times

Section 4: Reflection

Q8. What is Reflection in Java?

Ability to **inspect and manipulate** classes, methods, fields, and constructors at runtime — even private ones. Used by Spring, Hibernate, JUnit.

```
Class<?> clazz = Class.forName("java.lang.String");
Method m = clazz.getMethod("length");
Object result = m.invoke("Hello"); // 5

// Access private field
Field f = MyClass.class.getDeclaredField("secret");
f.setAccessible(true); // bypass private
Object val = f.get(myObj);
```

Q9. Drawbacks of Reflection?

Performance — slower than direct calls · **Security** — breaks encapsulation · **Complexity** — no compile-time checks · **Maintenance** — errors only at runtime

Section 5: Important Keywords

Keyword	Description
instanceof	Checks if object is instance of class/interface. Returns boolean. null instanceof X = false.
transient	Excludes field from serialization.
native	Method implemented in native C/C++ code via JNI.
strictfp	Ensures consistent IEEE 754 floating-point across platforms.
assert	Tests boolean condition during debugging. Throws AssertionError if false. Enable with -ea flag.

Section 6: Modern Java Features

Q10. What are Records? (Java 16)

Concise immutable data classes. Auto-generates constructor, getters, `equals()`, `hashCode()`, `toString()`.

```
record Person(String name, int age) { }
Person p = new Person("Alice", 25);
p.name(); // Alice
p.age(); // 25
System.out.println(p); // Person[name=Alice, age=25]
```

Q11. What are Sealed Classes? (Java 17)

Restrict which classes can extend or implement them. Better control over inheritance hierarchy.

```
sealed class Shape permits Circle, Rectangle, Triangle { }
final class Circle extends Shape { }
// class Pentagon extends Shape { } // Compile error!
```

Q12. What is pattern matching for switch? (Java 21)

```
String result = switch (obj) {  
    case Integer i -> "Integer: " + i;  
    case String s  -> "String: " + s;  
    case null      -> "Null value";  
    default        -> "Other: " + obj;  
};
```

All Topics Covered!

20 Topics · 500+ Questions · Complete Java Guide

Best of luck in your interviews and exams!